Fall 2026 DRP: Formal Languages

Michael Lee

I plan to keep these updated as the semester goes along, to serve as a record of the material and problems we've covered. Let me know if you spot any typos. These draw heavily on the lecture notes by Professor Timothy Ng, but all errors are mine alone.

Exercises are denoted  ,  ,  in increasing order of difficulty. We add  for exercises we've covered.

Contents

1 Introduction: Theory of Computation (Sep 9th) 1

2 Mathematical Induction (Sep 14th) 3

3 Strings 5
3.1 Languages 6

4 Deterministic Finite Automata (Sep 21st) 8
4.1 Closure properties of recognizable languages (Oct 5th) 12

5 Nondeterminism (Oct 12th) 14

1 Introduction: Theory of Computation (Sep 9th)

This is a project on computer science as a branch of mathematics. Math is the study of different kinds of structures, for example

- number theory is about the structure of the integers and their divisibility.
- combinatorics is about the enumeration of discrete structures
- calculus is about the properties of continuous structures, which interact with differentiation and integration operators.

What do we study in computer science? At the risk of sounding meta, we study the structures of *problems*, and the *algorithms* which solve them. But what does it mean to study the structure of problems?

In an intro programming course like CS 312, we usually think of problems as functions from some input to some output e.g. we compute the factorial function. In this setting, however, we will separate the definition of a function from the algorithm that computes it.

Definition 1.1. A function $f : A \to B$ is a binary relation i.e. $f \subset A \times B$ such that for every $a \in A$, there exists exactly one $b \in B$ such that $(a, b) \in f$ (we usually write this as $f(a) = b$.)

Two important observations:

- There's nothing in this definition about the algorithm on how to produce the required b given a; we only know that it exists.
- A function is itself a kind of set. Indeed, $f \subset A \times B$. Therefore, we will interchange between functions and sets based on whichever is easier to work with.

Similarly, given a definition of set, there might be sometimes no algorithm to tell us whether an element is a member of the set or not. For example, one very famous such set, called Fermat's last theorem, is the following

$$\{(a, b, c) \in \mathbb{Z}^3 : \exists n \geq 3, a^n + b^n = c^n\}$$

In this DRP, we will try to study what kinds of sets admit an algorithm that can check if a given object is an element of the said set.

Remark 1.2. *In computer science, problems are sets of strings.*

Remember that depending on your preferred programming languages, functions can take arrays, lists, dictionaries, even nested arrays (i.e. arrays of arrays), sets, floating points, memory pointers, etc. These can all be encoded by strings. To simplify our problem, we study the setting of string functions with binary outputs

$$P : \text{Strings} \to \{\text{Yes}, \text{No}\}$$

which give rise to an equivalent set

$$\{x \in \text{Strings} : P(x) = \text{Yes}\}.$$

So, we can phrase our question as:

Question 1.3. *How do we determine whether a given string lies in some defined set?*

Definition 1.4. Sets of strings are called languages.

This definition is not surprising considering the connections we will see to linguistics.

If all of this sounds terribly abstract, don't worry because we will revisit these concepts in due time after many examples. In a nutshell, any computer process consists at its core of just elaborate sequences of 1s and 0s. We want to understand the capabilities and limits of these bit operations.

Let's start working with some concrete strings.

2 Mathematical Induction (Sep 14th)

(adapted from MATH 161 UChicago Script 0) First, a detour into induction. We let $\mathbb{N}$ denote the natural numbers. In set notation,

$$\mathbb{N} = \{0, 1, 2, \dots, n, \dots\}.$$

Induction is a way to prove that a statement holds for all the natural numbers.

Theorem 2.1. *For each natural numbers n, let $P(n)$ be a proposition. Suppose we show the following two results:*

(a) $P(c)$ is true.

(b) For each natural number k, if $P(k)$ is true, then $P(k+1)$ is also true.

Then $P(n)$ is true for all natural numbers $n \geq c$.

We call part (a) the base case and part (b) the inductive step. The assumption that $P(k)$ is true is called the inductive hypothesis.

Example 2.2. *Let's use induction to prove that for all $n \geq 1$,*

$$\sum_{i=1}^{n} i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

Here's a video going over this example: `https://www.youtube.com/watch?v=wblW_M_HVQ8`.

Proof. For this problem, $P(n)$ denotes the property that

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

is true. First, we check $P(c)$ for the base case i.e. $c = 1$. We have

$$\sum_{i=1}^{1} i = 1 = \frac{1(1+1)}{2}.$$

Second, we carry out the inductive step. Suppose that we have the inductive hypothesis $P(k)$:

$$\sum_{i=1}^{k} i = \frac{k(k+1)}{2}.$$

Then

$$\begin{aligned}\sum_{i=1}^{k+1} i &= k+1+\sum_{i=1}^{k} i \\ &= k+1+\frac{k(k+1)}{2} \\ &= (k+1)\left(1+\frac{k}{2}\right) \\ &= (k+1)\frac{k+2}{2} \\ &= \frac{(k+1)(k+2)}{2},\end{aligned}$$

which confirms that $P(k+1)$ would also be true. Therefore, the statement holds for all $n \geq 1$. □

Example 2.3. *We can use induction prove different kinds of statements. For instance, we can use induction to show that $n \leq 2^n$ for all $n \geq 1$.*

Proof. For the base case, we check $1 \leq 2^1$. For the inductive step, suppose that we have $k \leq 2^k$. Then

$$k+1 \leq 2^k+1 \leq 2^k+k \leq 2^k+2^k \leq 2^{k+1},$$

which closes the induction. □

See

- `https://www.mathsisfun.com/algebra/mathematical-induction.html`

for more examples. Use induction for the following problems.

Exercise 2.1.  Prove that for all $n \geq 1$,

$$1^2+2^2+3^2+\cdots+n^2 = \sum_{i=1}^{n} i^2 = \frac{n(2n+1)(n+1)}{6}.$$

Exercise 2.2.  Prove that for all $n \geq 1$,

$$1+2+2^2+\cdots+2^n = 2^{n+1}-1.$$

Exercise 2.3.  Prove that for all $n \geq 1$, n^3-n is divisible by 3.

Exercise 2.4.  Prove that if $x > -1$ where $x \in \mathbb{R}$, then

$$(1+x)^n \geq 1 + nx$$

for any $n \geq 1$ where $n \in \mathbb{N}$.

Exercise 2.5.  Prove that if $n \geq 4$, then $n^2 \leq 2^n$.

Exercise 2.6.   Prove that if A is a non-empty subset of $\mathbb{N}$, then A has a least element, i.e. there is some $n_0 \in A$ such that for all $n \in A$ we have $n_0 \leq n$.

3 Strings

Definition 3.1. An alphabet is a finite set of symbols, usually denoted Σ.

Definition 3.2. A string $w = a_1 a_2 \dots a_n$ over an alphabet Σ is a finite sequence of symbols where each $a_i \in \Sigma$. The set of all strings over Σ is denoted Σ^*.

If you have seen recursion in computer science, we may instead define them this way

Definition 3.3. A string w over an alphabet Σ is either

- the empty string, denoted ϵ, or
- ax where $a \in \Sigma$ and $x \in \Sigma^*$ is a string.

Definition 3.4. The length of a word w is denoted by $|w|$ and is the number of symbols in w. Inductively,

- if $w = \epsilon$, then $|w| = 0$, and
- if $w = ax$ for $a \in \Sigma, x \in \Sigma*$, then $|w| = 1 + |x|$.

Definition 3.5. The concatenation of two strings $u = a_1 a_2 \dots a_m$ and $v = b_1 b_2 \dots b_n$ is $uv = a_1 \dots a_m b_1 \dots b_n$. Inductively, we have

- If $u = \epsilon$, then $uv = \epsilon v = v$, or
- If $u = ax$ for $a \in \Sigma, x \in \Sigma^*$, then $uv = a(x \cdot v)$.

Remark 3.6. *String concatenation is associative but not commutative.*

Definition 3.7. Let Σ and Γ be two alphabets. A string homomorphism is a function $h : \Sigma^* \to \Gamma^*$ such that for all $u, v \in \Sigma^*$, $h(uv) = h(u)h(v)$.

Definition 3.8. The kth power of a string w is $w^k = \overbrace{w \dots w}^{k \text{ times}}$. Inductively

- $w^0 = \epsilon$, and
- $w^k = w^{k-1}w$.

Definition 3.9. If $w = uxv$ is a string, we say u is a prefix, v is a suffix, and x is a substring of w.

Question 3.10. *What are the prefixes, suffices and substrings of "cat"?*

Definition 3.11. The reversal of a string w is defined by

- if $w = \epsilon$, then $\epsilon^R = \epsilon$, and
- if $w = ax$ for a symbol $a \in \Sigma$ and a string $x \in \Sigma^*$, then $(ax)^R = x^R a$.

3.1 Languages

Definition 3.12. A language L over Σ is a subset $L \subset \Sigma^*$.

Some examples

- $\{w \in \{a, b\}^* : |w|_a \geq |w|_b\}$
- $\{w \in \Sigma^* : w = w^R\}$,
- $\{w \in \{0, 1\} : w$ is the base-2 representation of a prime number$\}$
- $\{w \in \{A, C, G, T\}^* : ACC$ is not a substring of $w\}$.

Definition 3.13. The complement of L is $\overline{L} = \Sigma^* \setminus L$.

Definition 3.14. Let L_1, L_2 be two languages. We define the concatenation by

$$L_1 L_2 = \{uv : u \in L_1, v \in L_2\}.$$

Just like string concatenation, language concatenation is not commutative.

Definition 3.15. The kth power of a language is

- $L^0 = \{\epsilon\}$, and
- $L^k = L^{k-1}L$.

Definition 3.16. The Kleene star of L is defined by

$$L^* = \bigcup_{i \geq 0} L^i.$$

We define the positive Kleene closure of L by $L^+ = LL^*$.

Exercise 3.1.  Let $w \in \Sigma^*$. Show that $(w^R)^i = (w^i)^R$ for all $i \in \mathbb{N}$.

Exercise 3.2.  Let $x, y \in \Sigma^*$. Show that $|xy| = |x| + |y|$.

Exercise 3.3.  Let $A, B, C \subset \Sigma^*$ be languages. Prove that $(A \cup B)C = AC \cup BC$.

Exercise 3.4.  Let $u, v \in \Sigma^+$. Show that if $(uv)^i u = u(vu)^i$ for all $i \in \mathbb{N}$.

Exercise 3.5.  Let $\Sigma = \{a, b\}$ and $Z_1 = \{w \in \Sigma^* : |w|_a = |w|_b\}$. Prove that

$$\mathrm{Pre}(Z_1) = \{a, b\}^*.$$

Exercise 3.6.  Let $u, v, x, y \in \Sigma^*$ and suppose that $uv = xy$. Prove that if $|u| \geq |x|$, then there exists $t \in \Sigma^*$ such that $u = xt$ and $y = tv$.

Exercise 3.7.  Let $L \subset \Sigma^*$ be a language. Show that $L^2 \subset L$ if and only if $L = L^+$.

Exercise 3.8. 

(a) Given a homomorphism $h : \Sigma^* \to \Sigma^*$, natural number $n \in \mathbb{N}$ and $w \in \Sigma^*$, give a formal inductive definition for $h^n(w)$.

(b) Prove that for all homomorphisms $h : \Sigma^* \to \Delta^*$ and natural numbers n such that $h^n(uv) = h^n(u)h^n(v)$ for all strings $u, v \in \Sigma^*$.

(c) Let $\Sigma = \{0, 1\}$. Consider the sequence of strings $\{X_n\}_{n \geq 0}$, defined by

- $X_0 = 0$,
- $X_1 = 01$, and
- $X_n = X_{n-1}X_{n-2}$ for $n \geq 2$.

Let φ be the homomorphism satisfying $\varphi(0) = 01, \varphi(1) = 0$. Prove that $X_n = \varphi^n(0)$ for all $n \in \mathbb{N}$.

Exercise 3.9.  Let L and M be languages over Σ. Prove that $(L^*M^*)^* = (L \cup M)^*$.

4 Deterministic Finite Automata (Sep 21st)

We are finally ready to tackle our first machines. These are Turing machines with finitely many states, finitely many instructions and no ability to write to the string. In addition, they can only parse a string from left to right and move one step at each time.

Example 4.1. *Let's go over a DFA that checks if a string contains cat as a substring.*

I used the website `https://madebyevan.com/fsm/` to create the following diagram. It is also possible to use the `automata` package in Python to get

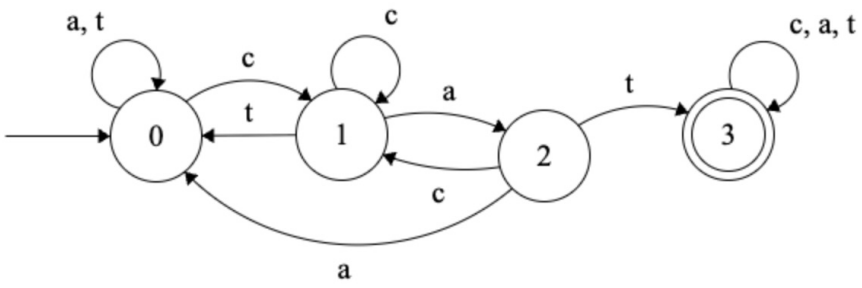


Figure 1: State diagram

Definition 4.2. A deterministic finite automaton is a 5-tuple $A = (Q, \Sigma, \delta, q_0, F)$, where

- Q is a finite set of states,
- Σ is the input alphabet,
- $\delta : Q \times \Sigma \to Q$ is a transition function,
- q_0 is the initial state,
- F is a set of final/accepting states.

We can represent the transition function for fig. 1 with a table like so:

	a	c	t
0	0	1	0
1	2	1	0
2	0	1	3
3	3	3	3

We can also run the following code in a Google Colab notebook.

```
import sys
!apt-get update -qq > /dev/null
!apt-get install -y graphviz libgraphviz-dev pkg-config > /dev/null
!{sys.executable} -m pip install coloraide pygraphviz
!{sys.executable} -m pip install automata-lib
from automata.fa.dfa import DFA
import itertools

# This the machine that accepts strings that contain cat as a substring.
A = DFA(
    states=set('0123'),
    input_symbols=set('act'),
    transitions={
        '0': {'a': '0', 'c': '1', 't': '0'},
        '1': {'a': '2', 'c': '1', 't': '0'},
        '2': {'a': '0', 'c': '1', 't': '3'},
        '3': {'a': '3', 'c': '3', 't': '3'}
    },
    initial_state='0',
    final_states={'3'}
)
```

In the spirit of inductive definitions, we can take a given transition function $\delta : Q \times \Sigma \to Q$ to get a transition function that takes a string instead of a letter $\delta^* : Q \times \Sigma^* \to Q$.

Definition 4.3. Given $\delta : Q \times \Sigma \to Q$, we define $\delta^* : Q \times \Sigma^* \to Q$ by

- $\delta^*(q, \epsilon) = q$,
- $\delta^*(q, ax) = \delta^*(\delta(q, a), x)$ or alternatively, $\delta^*(q, xa) = \delta(\delta^*(q, x), a)$.

It is not hard to see that we get $\delta^*(q, uv) = \delta^*(\delta^*(q, u), v)$. This definition is useful because we can later break strings into pieces.

Definition 4.4. A DFA A accepts a string $w \in \Sigma^*$ when $\delta^*(q_0, w) \in F$. We say that the language of A is

$$L(A) = \{w \in \Sigma^* : \delta^*(q_0, w) \in F\}.$$

Remark 4.5. *Let A be given as in fig. 1. Let $L_{cat} = \{w \in \{a, c, t\}^* : \text{"cat" is a substring of } w\}$. Check that $L(A) = L_{cat}$.*

Definition 4.6. We say that a language $L \subset \Sigma^*$ is recognizable if it is accepted by a DFA A.

Example 4.7. *Let*

$$L_{\text{div } 3} = \{w \in \{0, 1\}^* : (|w|)_2 \text{ is divisible by } 3\},$$

where $(|w|)_2$ *is the integer represented by* w *in base* 2.

The crucial observation in this case is that $(|xa|)_2 = 2(|x|)_2 + a$*, so we want the following transition table*

	0	1
0	0	1
1	2	0
2	1	2

The corresponding state diagram looks like

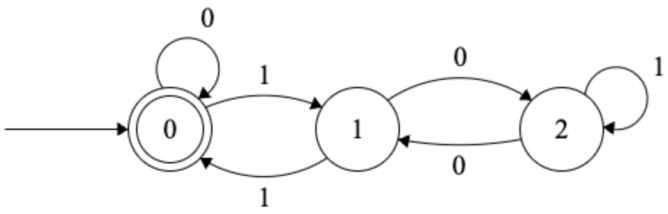


Figure 2: State diagram

Proposition 4.8. $L(A) = L_{\text{div}\,3}$*, i.e. the language of strings accepted by the DFA* A *is precisely the set of strings they represent a number divisible by* 3.

Lemma 4.9. *For all strings* $w \in \{0,1\}^*$ *and states* $q \in \{0,1,2\}, \delta^*(0,w) = q$ *if and only if* $(|w|)_2 = q \pmod 3$.

Proof. We prove by induction on w.

For the base case, we have $\delta^*(0,\epsilon) = 0$ and $(|\epsilon|)_2 = 0 \pmod 3$.

Now let $w = xa$ with $a \in \Sigma$ and $x \in \Sigma^*$. By the inductive hypothesis, we have $\delta^*(0,x) = p$ if and only if $(|x|) = p \pmod 3$. Then

$$\begin{aligned}
\delta^*(0,xa) &= \delta(\delta^*(0,x),a) \\
&= \delta((|x|_2 \pmod 3), a) \\
&= (2(|x|)_2 \pmod 3 + a) \pmod 3 \\
&= (2(|x|)_2 + a) \pmod 3 \\
&= (|xa|)_2 \pmod 3.
\end{aligned}$$

Then the following are equivalent

- $w \in L_{\text{div }3}$,
- $3 \mid (|w|)_2$,
- $(|w|)_2 = 0 \pmod 3$,
- $\delta^*(0, w) = 0$,
- $w \in L(A)$.

□

This argument accomplishes two steps: we have to check that every string in $L_{\text{div }3}$ is accepted by A, *and* every string in the complement is rejected by A.

It is possible to extend this construction to any integer n: we can construct a DFA that recognizes all of the binary strings that represent integers that are divisible by n.

Another key observation is that even though $L_{\text{div }3}$ is an infinite language, it can be encapsulated with a finite amount of information, which allows us to solve it with a DFA.

Exercise 4.1.  Construct a DFA for the set of strings over $\{a, b\}$ that end in aa.

Exercise 4.2.  Construct a DFA for the set of strings over $\{a, b\}$ that contain three consecutive a's.

Exercise 4.3.  Construct a DFA for the set of string over $\{a, b, c\}$ such that all a's precede all b's and all b's precede all c's.

Exercise 4.4.  Construct a DFA for the binary strings that represent integers divisible by n.

Exercise 4.5.  Construct a DFA for the set of strings over $\{a, b\}$ that contain an odd number of as and at most four bs.

Exercise 4.6.  Let $\Sigma = \{a, b\}$ and let $Z_2 = \{w \in \{a, b\}^* : |w|_{ab} = |w|_{ba}\}$. In other words, Z_2 is the language of strings such that w contains the substring ab as many times as it does the string ba.

Construct a DFA A that accepts Z_2.

(a) Give a state diagram for your DFA A.

(b) Explain why $L(A) = Z_2$.

Exercise 4.7.  Construct a DFA for the language

$$L_{\neg aba} = \{w \in \{a,b\}^* : w \text{ does not contain the substring } aba\}.$$

(a) Give a state diagram for your DFA A.

(b) Explain why $L(A) = L_{\neg aba}$.

4.1 Closure properties of recognizable languages (Oct 5th)

Recall the Boolean operations on sets given by: complement, union, and intersection. A natural question to ask is the following: if a language L is recognizable by some DFA A, can we construct a new DFA that recognizes its complement?

Definition 4.10. We say that a class of languages is closed under an operation if whenever the arguments of the operation are in the class, the result is also.

Theorem 4.11. *The recognizable languages are closed under complement.*

Definition 4.12. Consider DFAs $A_1 = (Q_1, \Sigma, \delta_1, q_{0,1}, F_1)$ and $A_2 = (Q_2, \Sigma, \delta_2, q_{0,2}, F_2)$. Then the product automaton is the DFA $A' = (Q', \Sigma, \delta', q_0', F')$, where

- $Q' = Q_1 \times Q_2$,
- $\delta'(\langle p, q\rangle, a) = \langle \delta_1(p,a), \delta_2(q,a)\rangle$, where $p, \in Q_1, q \in Q_2, a \in \Sigma$,
- $q_0' = \langle q_{0,1}, q_{0,2}\rangle$,
- $F' = \{\langle p, q\rangle \mid p \in F_1, q \in F_2\} = F_1 \times F_2$.

Lemma 4.13. *For all $w \in \Sigma^*$,*

$$\delta'^*(\langle p, q\rangle, w) = \langle \delta_1^*(p,w), \delta_2^*(q,w)\rangle.$$

Theorem 4.14. $L(A') = L_1 \cap L_2$.

Corollary 4.15. *The class of recognizable languages is closed under intersection and union.*

Exercise 4.8.  (Sipser 1.5) Construct a DFA for the given language, under the alphabet $\Sigma = \{a, b\}$. Hint: Use theorem 4.11.

(a) $\{w : w \text{ does not contain the substring } ab\}$

(b) $\{w : w \text{ does not contain the substring } baba\}$

(c) $\{w : w \text{ contains neither the substrings } ab \text{ nor } ba\}$

(d)  $\{w : w$ is any string not in $\{a\}^*\{b\}^*\}$

(e) $\{w : w$ is any string not in $\{ab\}^*\}$

(f) $\{w : w$ is any string not in $\{a\}^* \cup \{b\}^*\}$

(g) $\{w : w$ is any string that doesn't contain exactly two a's$\}$

Exercise 4.9.  (Sipser 1.4) Construct a DFA for the given language, under the alphabet $\Sigma = \{a, b\}$. Hint: Use theorem 4.15.

(a) $\{w : w$ has at least three a's and at least two b's $\}$

(b) $\{w : w$ has at exactly two a's and at least two b's $\}$

(c) $\{w : w$ has an even number of a's and one or two b's $\}$

(d) $\{w : w$ has an even number of a's and each a is followed by at least one b $\}$

(e) $\{w : w$ starts with an a and has at most one b $\}$

(f) $\{w : w$ has an odd number of a's and ends with a $b\}$

(g) $\{w : w$ has even length and an odd number of a's$\}$

Exercise 4.10.  Let A be a DFA. Construct a DFA that recognizes the language

$$\mathrm{Pre}(A) = \{u \in \Sigma^* : \exists v \in \Sigma^* : uv \in L(A)\}.$$

Exercise 4.11.  For two languages L_1 and L_2 over the same alphabet Σ, we define the *prefix mask* operation $\preccurlyeq$ by

$$L_1 \preccurlyeq L_2 = \{uv \mid uv \in L_1 \wedge u \in L_2\}.$$

Given DFAs A_1 and A_2, construct a DFA $A_{\preccurlyeq}$ that accepts the language $L(A_1) \preccurlyeq L(A_2)$. Describe how your machine works informally and show that it is correct.

Exercise 4.12.  Let $L_1, L_2 \subset \Sigma^*$ be languages. We define the cut operation by

$$L_1 \Cap L_2 = \{uv : u \in L_1, v \in L_2, \forall v' \in \mathrm{Pre}(v) \setminus \{\epsilon\}, uv' \notin L_1\}.$$

That is, $L_1 \Cap L_2$ is the set of all strings uv that are in L_1L_2 subject to the condition that there is no non-empty prefix v' of v such that uv' is a string in L_1. Informally, one can interpret this as an operation that picks the strings where strings from L_1 are as long as

possible – a "greedy" match of the strings in that language.

For example, if $L_1 = \{a^n : n \in \mathbb{N}\}$ and $L_2 = \{ac, bc\}$ then $L_1 \Cap L_2 = \{a^n bc : n \geq 1\}$. Why? Consider the possible options for uv with $u \in L^1$ and $v \in L_2$. The string u is of the form a^n. Then we have either $a^n ac$ or $a^n bc$. Since $a^n b$ and $a^n bc$ are both not in $L_1, a^n bc \in L_1 \Cap L_2$. However, $a^n a = a^{n+1}$ is a prefix of $a^n ab$ which is in L_1, so strings $a^n ab$ are not in $L_1 \Cap L_2$.

Given DFAs A_1, A_2, construct a DFA A' that recognizes the language $L(A_1) \Cap L(A_2)$ and describe informally why the DFA is correct.

5 Nondeterminism (Oct 12th)

Having seen that the recognizable languages are closed under boolean operations, we want to ask the same question about other kinds of operations. Recall the concatenation operation on languages

$$L_1 L_2 := \{uv : u \in L_1, v \in L_2\}.$$

For this purpose, we define a (new?) kind of machine as follows.

Definition 5.1. A non-deterministic finite automaton (NFA) is a 5-tuple $A = (Q, \Sigma, \delta, q_0, F)$.

- Similar to the DFA, Q represents the states, Σ the alphabet, q_0 the initial state, F the set of final states.
- However, the codomain of δ is now $\mathcal{P}(Q)$ i.e. $\delta : Q \times \Sigma \to \mathcal{P}(Q)$.

Remark 5.2. *δ does not have to be a total function. It is possible for $\delta(q, a) = \emptyset$.*

We alter the acceptance condition to allow at least one "branch" to end up in a final state. Analogously to theorem 4.3, we have

Definition 5.3. $\delta^* : Q \times \Sigma^* \to \mathcal{P}(Q)$ is given by

- $\delta^*(q, \epsilon) = \{q\}$,
- $\delta^*(q, xa) = \bigcup_{p \in \delta^*(q,x)} \delta(p, a)$.

Definition 5.4. The language of strings accepted by an NFA A is given by

$$L(A) = \{w \in \Sigma^* : \delta^*(q_0, w) \cap F \neq \emptyset\}.$$

Theorem 5.5. *If L_1, L_2 are recognizable, there exists an NFA that accepts $L_1 L_2$.*

Proof. Take $A' = (Q', \Sigma, \delta', q_0', F')$, where $Q' = Q_1 \sqcup Q_2, q_0' = q_{0,1}, F' = F_2$. We take

$$\delta'(q,a) = \begin{cases} \{\delta_1(q,a)\} & q \in Q_1 \text{ and } \delta_1(q,a) \notin F, \\ \{\delta_2(q,a)\} & q \in Q_2, \\ \{\delta_1(q,a), q_{0,2}\} & q \in Q_1 \text{ and } \delta_1(q,a) \in F. \end{cases}$$

We check that $w \in L(A')$ if and only if $w \in L_1, L_2$. Indeed, we can write $w = xy$ such that $\delta_1(q_{0,1}, x) \in F_1$ and $\delta_2(q_{0,2}, y) \in F_2$. □

Exercise 5.1.  Construct an NFA for the set of strings over $\{a, b\}$ that have the same third last character as its first character.

Exercise 5.2.  Construct an NFA for the set of strings over $\{a, b\}$ that contain $abaab$ as a substring.

Exercise 5.3.  Let A be a DFA. Construct an NFA that recognizes the language

$$\mathrm{Suf}(A) = \{u \in \Sigma^* : \exists v \in \Sigma^* : vu \in L(A)\}.$$

Exercise 5.4.  Let $\Sigma_k = \{1, 2, \ldots, k\}$. We define the language $L_{\neg k}$ by

$$L_{\neg k} = \{w \in \Sigma_k^* \mid \exists i \in \Sigma_k, w \in (\Sigma_k \setminus \{i\})^*\}.$$

(a) Construct an NFA A that accepts $L_{\neg k}$. Describe how your machine works informally. How many states does A have?

(b) Construct a DFA B that accepts $L_{\neg k}$. Describe how your machine works informally. How many states does B have?

Exercise 5.5.  Let $w \in \Sigma^*$ be a string and $L \subset \Sigma^*$ be a language. The quotient of L by w is defined by

$$w^{-1}L = \{x \in \Sigma^* : wx \in L\}.$$

For example, if $L = \{abaabaaba, aaaa\}$, then

- $(aba)^{-1}L = \{abaaba\}$,
- $(aaaa)^{-1}L = \{\epsilon\}$, and
- $(abb)^{-1}L = \emptyset$.

We can extend quotients to languages by

$$L_1^{-1}L_2 = \{x \in \Sigma^* : \exists w \in L_1, wx \in L_2\}.$$

Given a DFA A and a finite language $L = \{w_1, \ldots, w_n\}$, construct an NFA A' that accepts the language $L^{-1}L(A)$.

Exercise 5.6.  Let $w = a_1 \dots a_n$ be a string of length n over Σ. We define the duplication operation by

$$\Rightarrow w = \{uxxv : \exists u, v, x \in \Sigma^*, w = uxv\}.$$

That is, the duplication operation on a string w produces a set of strings where some substring of w is repeated.

Given a string, define an NFA that accepts the language $\Rightarrow w$.

Exercise 5.7.  For strings u and v, we define the *insertion* operation by

$$u \leftarrow v = \{u_1 v u_2 \mid u = u_1 u_2\}.$$

For example,

$$hot \leftarrow dog = \{doghot, hdogot, hodogt, hotdog\}.$$

Then the insertion of a language L_2 into a language L_1 is defined

$$L_1 \leftarrow L_2 = \bigcup_{u \in L_1, v \in L_2} u \leftarrow v.$$

Let A_1 and A_2 be DFAs. Construct an NFA B that accepts the language $L(A_1) \leftarrow L(A_2)$. Explain how your machine works informally and show that it is correct (i.e. that $L(B) = L(A_1) \leftarrow L(A_2)$).